FROM COMMIT TO LIVE TICKET

PROJECT TEAM

Fahad Ullah
Samuel Udom
Suzlana Mboglem

NOV 2026

A secure, tested and observable ticketing platform

SOME OF OUR TECH STACK

TEAM B / TOOLCHAIN


Python


Docker


Jenkins


Prometheus


Grafana


Loki


SQL


AWS


Trivy


GitHub


HTML

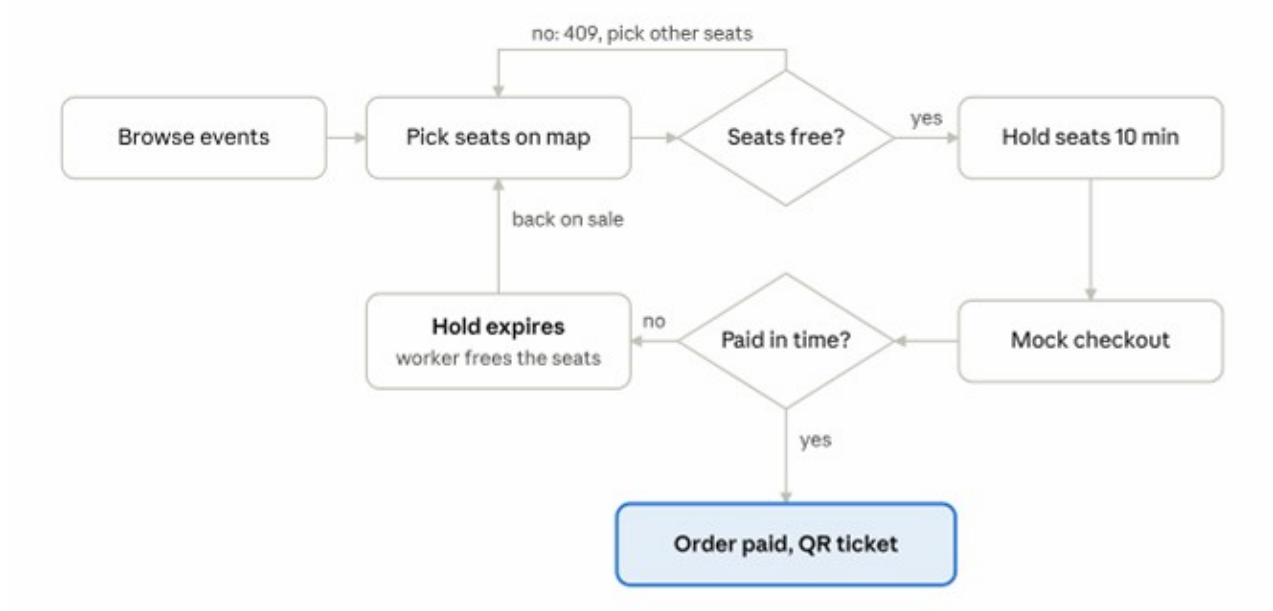

CSS

PROJECT BRIEF / INITIAL DESIGN / 06 NOV 2026

Initial System Design & Booking Flow

Original booking flow, core tech stack and service communication.

PLANNED BOOKING FLOW



PROJECT GOALS

Set out in the original brief

01 **CUSTOMER JOURNEY**
Exact seat, 10-minute hold, QR ticket

02 **NO DOUBLE-BOOKING**
Target: 200 users / 50 seats

03 **SAFE AWS DELIVERY**
Tests, CVE gate, smoke test + rollback

04 **VISIBLE + REPEATABLE**
Grafana alerts; Terraform rebuild

PLANNED GOALS • NOT TEST RESULTS

TEAM RESPONSIBILITY

01 / APPLICATION

SUZLANA MBOGLEM

- Flask API and PostgreSQL
- Events, seats, holds and checkout
- Double-booking protection
- Frontend and application tests

02 / INFRASTRUCTURE

SAMUEL UDOM

- Docker and Docker Compose
- Terraform: VPC, EC2, RDS, ECR
- Health checks and AWS secrets
- Staging and production setup

03 / CI/CD + MONITORING

FAHAD ULLAH

- Jenkins pipeline and Trivy scans
- Prometheus, Grafana and Loki
- Slack alerts and GitHub webhook
- Project structure and checkout API

PROJECT MANAGEMENT

Jira task ownership / Sprint dependencies / Gantt timeline

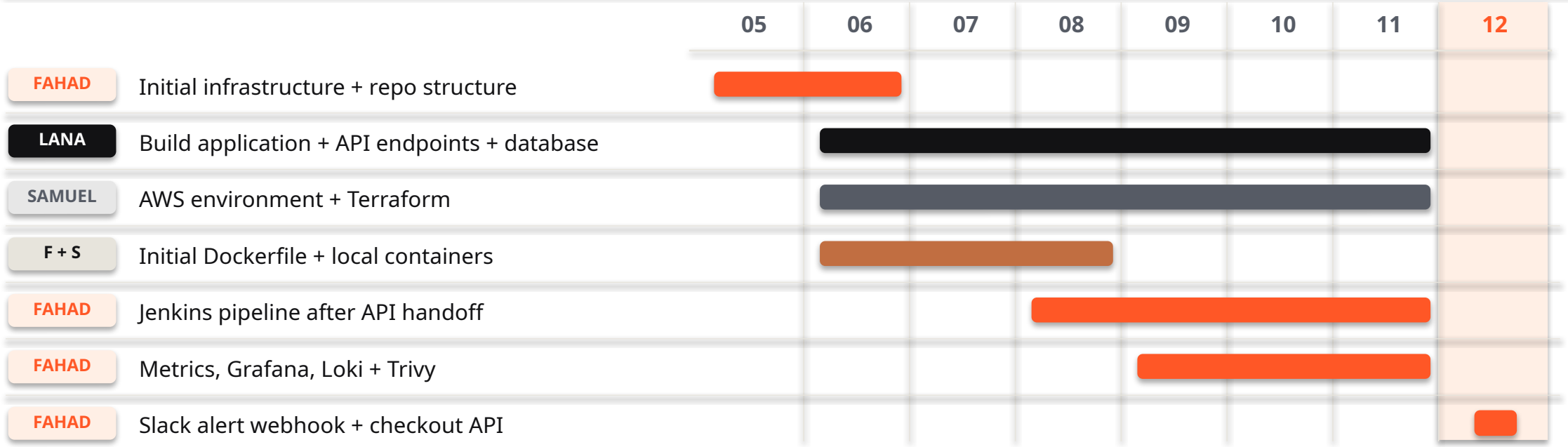
KICKOFF
05 NOV 2026

ORIGINAL DEADLINE
11 NOV 2026

ACTUAL FINISH
12 NOV 2026 +1 DAY

HOW WORK PROGRESSED

NOV 2026 / RECONSTRUCTED



THE PRODUCT PROMISE

Your seat, sorted.

A clear customer journey, with a protected seat at its centre.

01

BROWSE

Find an event

→

02

PICK SEAT

Choose exact seat

→

03

HOLD

10-minute reservation

→

04

TICKET

Digital ticket prototype

NO DOUBLE-BOOKING

Login, payment and ticket screens are prototype UI.

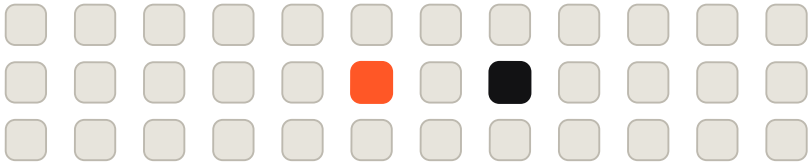
BOOKING INTEGRITY

Lock the seat before the sale.

Database transactions enforce the booking rules.

SELECT → ROW LOCK → 10-MIN HOLD → CHECKOUT → TICKET

SEAT INVENTORY



SELECT ... FOR UPDATE SKIP LOCKED

Unique event-seat ticket constraint. Expired holds are reusable when queried or retried; no expiry worker.

TARGET: 200 USERS / 50 SEATS / ZERO DUPLICATES

C460 TEAM B · USHR TICKETING APP

API & Database

How I made sure no seat is ever sold twice, and got it live on AWS.

Suztlana Mboglem
Person 1 · Flask API & PostgreSQL

STACK
Flask · SQLAlchemy · PostgreSQL / RDS · Docker · Jenkins · Prometheus

A database that refuses to double-sell

`venues`
The building

`seats`
Row F, seat 12

`events`
A show on a date

`event_seats`
Each seat's price and status for one show: available, held or sold. This is where booking happens.

`orders`
A purchase

`tickets`
One per seat, at most

Rules the database enforces itself

- **Foreign keys:** every seat belongs to a real venue
- **Unique:** one ticket per seat per show, the last line of defence
- **Checks:** status must be available, held or sold; no negative prices
- **Versioned migrations:** `flask db upgrade` builds the same schema everywhere
- **Seed data:** 60 seats, 2 events, 120 bookable seats

Ten buyers, one seat, one winner

1 request got `201 Held`

9 got `409 Taken`

Ten simultaneous requests for the same seat, against real PostgreSQL. Result captured on the next slide.

HOW A HOLD WORKS

1. Lock the seats you asked for: `SELECT … FOR UPDATE SKIP LOCKED`
2. Got them all? Mark them held for 10 minutes
3. Missing even one? Roll back, hold nothing, return 409
4. Expired holds count as free again automatically

Ten buyers, one seat: tested on AWS

```
viola@LAPTOP-0F4GHKAR MINGW64 ~/Documents/C460-TeamB (develop)
$ SEAT=55   # change to the id from step 2
for i in $(seq 1 10); do
  curl -s -o /dev/null -w "buyer $i -> %{http_code}\n" -X POST http://18.171.194.63/api/v1/events/1/holds \
    -H "Content-Type: application/json" \
    -d "{\"event_seat_ids\":[$SEAT],\"customer\":\"buyer$i@example.com\"}" &
done; wait
[1] 1889
[2] 1890
[3] 1891
[4] 1892
[5] 1893
[6] 1894
buyer 1 -> 201
buyer 2 -> 409
[7] 1895
buyer 3 -> 409[8] 1896

buyer 4 -> 409
[9] 1897
[10] 1898
buyer 5 -> 409[1]   Done                       curl -s -o /dev/null -w "buyer $i -> %{http_code}\n" -X POST
at_ids\":[$SEAT],\"customer\":\"buyer$i@example.com\"}"

buyer 6 -> 409
buyer 7 -> 409
buyer 8 -> 409
buyer 9 -> 409
buyer 10 -> 409[5]   Done                       curl -s -o /dev/null -w "buyer $i -> %{http_code}\n" -X POST
eat_ids\":[$SEAT],\"customer\":\"buyer$i@example.com\"}"

[10]+  Done                    curl -s -o /dev/null -w "buyer $i -> %{http_code}\n" -X POST http://18.171
AT],\"customer\":\"buyer$i@example.com\"}"
```

```
viola@LAPTOP-0F4GHKAR MINGW64 ~/Documents/C460-TeamB (develop)
$ curl -s http://18.171.194.63/api/v1/events/1/seats | grep -oE "\"event_seat_id\":$SEAT,[^}]*"
"event_seat_id":55,"number":5,"price_pence":3500,"row":"F","section":"Circle","status":"held"

viola@LAPTOP-0F4GHKAR MINGW64 ~/Documents/C460-TeamB (develop)
$ curl -s http://18.171.194.63/metrics | grep -E '^ticketing_(seat_holds_total|hold_requests_total|seats_held) |seats_unavailable"\} '
ticketing_seat_holds_total 1.0
ticketing_hold_requests_total 10.0
ticketing_hold_failures_total{reason="seats_unavailable"} 9.0
ticketing_hold_failures_created{reason="seats_unavailable"} 1.7945436644348478e+09
ticketing_seats_held 1.0
```

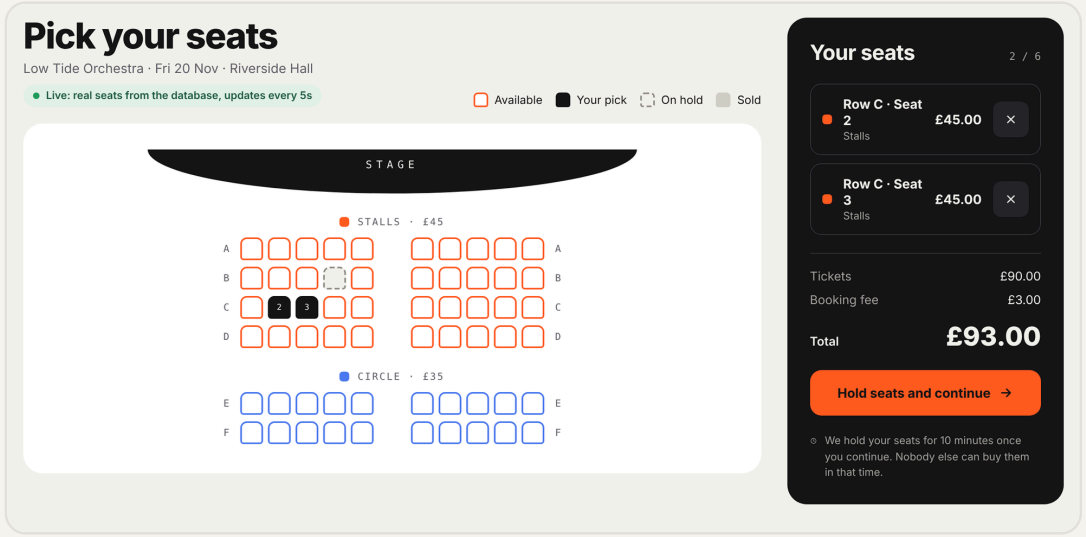
01 **10 requests, same seat, same instant.** Fired from my laptop at the live site.

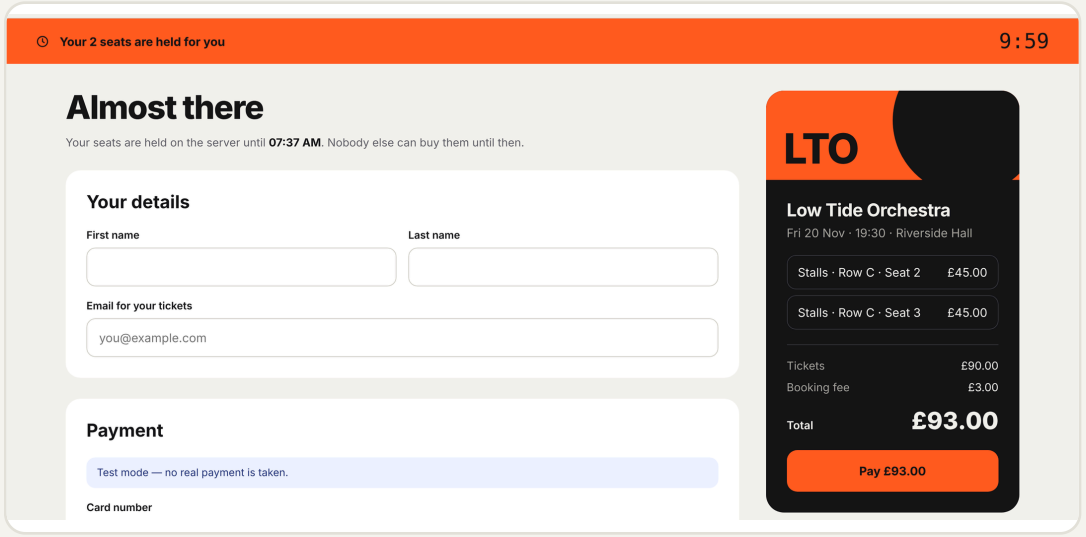
02 **1 × 201, 9 × 409.** One buyer got it; everyone else was refused straight away.

03 **Checked twice below:** the seat shows held once, and the metrics count 1 hold and 9 refusals.

Run from Git Bash against the live site, http://18.171.194.63, 13 Nov 2026

What a buyer sees





01 **Pick seats on the live map.** Prices and status come from the database and refresh every 5 seconds.

02 **Held for 10 minutes.** The server confirms the hold, and the countdown runs from the server's expiry time.

Captured from the real app and API against PostgreSQL 16, 13 Nov 2026

The API, and how we can see it working

Endpoints I built

Endpoint	What it does
`GET /api/v1/events`	Lists shows, soonest first
`GET …/events/<id>/seats`	Seat map with price and live status
`POST …/events/<id>/holds`	Holds seats: 201, or 409 if taken
`GET /healthz · /readyz`	App up · database reachable (503 within 3s if not)
`GET /metrics`	Numbers for Prometheus and Grafana

What /metrics reports

Traffic, errors, p95 latency
Per route, including 4xx and 5xx

Holds and booking failures
Failures split by reason: taken, invalid, not on sale

Seats held, expired, available
Plus database query time and a db-up flag

From my laptop to live on AWS

01
Container ready
Added migrations to the Docker image

02
Deploy builds the DB
Migrations run before the switch, skipped on rollback

03
RDS, no extra access
Schema created through Jenkins, least privilege

04
Website uses real data
Live seat map, real holds, real hold timer

Result

Jenkins build #28: lint, security scans, deploy and smoke test all green. Failed deploys roll back automatically.

Next steps

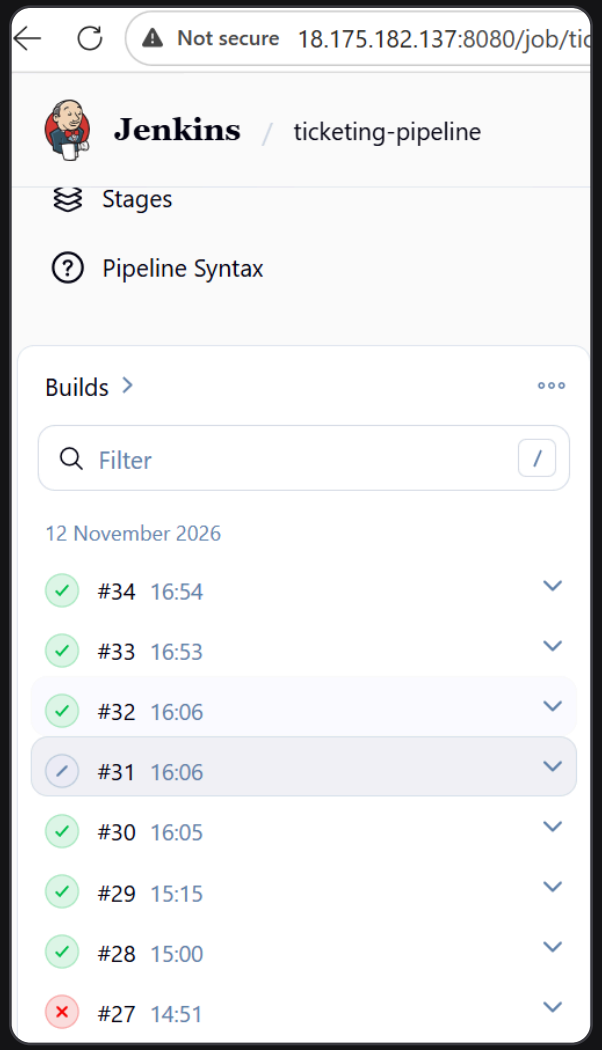
Checkout and real tickets · login with JWT · automated tests · auto-deploy on every push

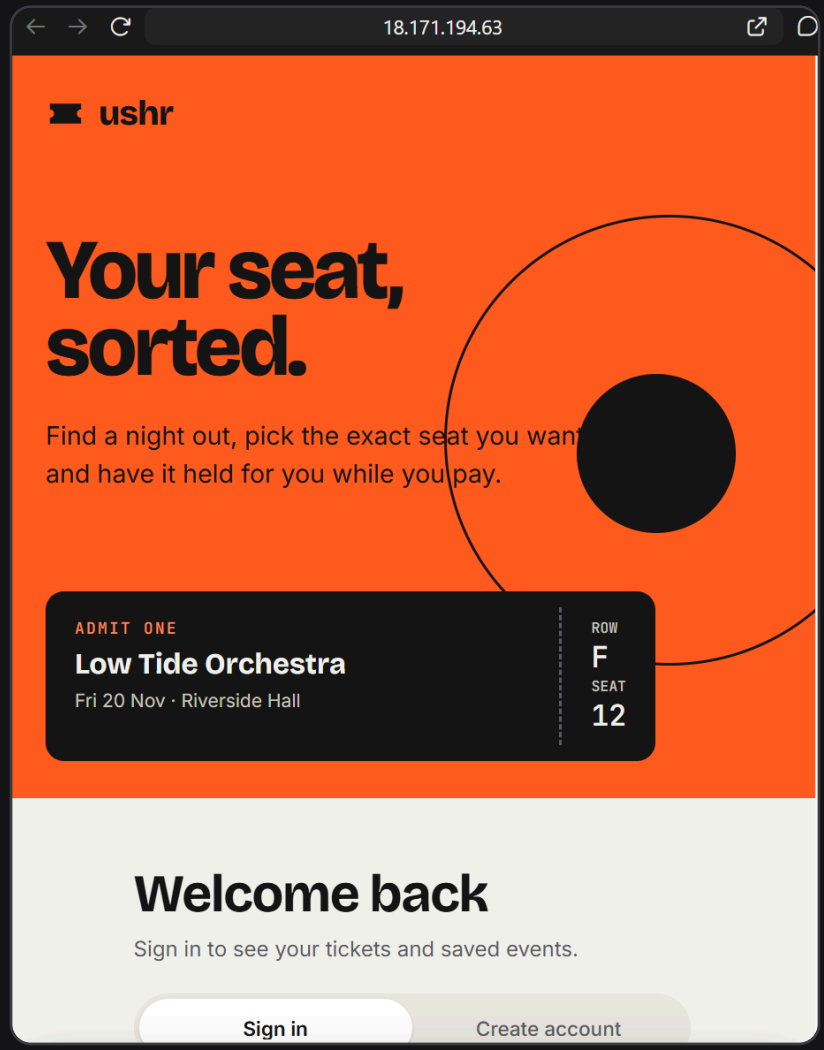
Site: `http://18.171.194.63`

Live on AWS

01 **Jenkins:** build #27 failed the lint stage. After my fix, #28 was the first fully green run: lint, security scans, deploy and smoke test.

02 **The site:** Ushr running on our EC2 server, with its data in RDS.





Site: `http://18.171.194.63`

VERSION CONTROL + DELIVERY PIPELINE

GitFlow Branching Strategy with Jenkins CI/CD

01 / GITFLOW-INSPIRED BRANCHING

feature/*
infra/*
fix/*

→ **develop**
INTEGRATION

MERGE →

main
STABLE BRANCH

02 / JENKINS CI/CD PIPELINE

COMMIT
GitHub

→ **VALIDATE**
Frontend + tests

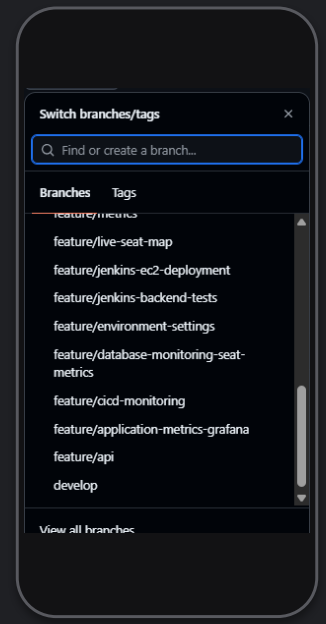
→ **PACKAGE**
Docker + Trivy

→ **RELEASE***
ECR + SSM

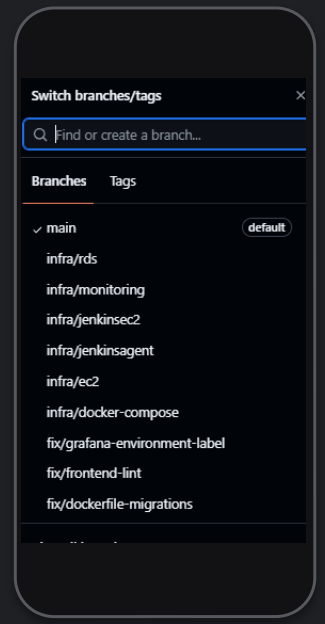
→ **VERIFY**
/readyz + rollback

CI: tests + security checks | CD: AWS release only when DEPLOY_TO_AWS is enabled

REAL GITHUB BRANCHES



FEATURE / DEVELOP



INFRA / FIX / MAIN

Branch names shown; merge and deployment results require run evidence.

FAHAD ULLAH / CI PIPELINE

Jenkins pipeline & Trivy scanning

Code checkout, frontend validation, backend tests and repository security scanning

JENKINS STAGES / MY WORKSTREAM

STAGE VIEW

01 ✓
CHECKOUT
GitHub workspace

02 ✓
VALIDATE STRUCTURE
Project files

03 ✓
INSTALL FRONTEND
npm ci

04 ✓
FRONTEND LINT
HTML / CSS / JS

05 ✓
FRONTEND VALIDATION
Structural checks

CONTINUED / TEST • SCAN • PACKAGE

06 ✓
BACKEND TESTS
Unit + integration

07 ✓
TRIVY REPO SCAN
Source dependencies

08 ✓
BUILD IMAGE
Docker build

09 ✓
POST ACTIONS
Pipeline wrap-up

LOCAL JENKINS

Docker controller • localhost:8081 • persistent volume

FAHAD ULLAH / OBSERVABILITY

Prometheus monitoring

Centralised metrics collection for the application and supporting services

METRICS COLLECTION / 15-SECOND INTERVAL

SCRAPE ARCHITECTURE / FLASK BACKEND

FLASK BACKEND

/metrics

/healthz
/readyz

→

PROMETHEUS

15s

scrape interval

→

GRAFANA

Dashboards

Visualisation

HTTP Requests • 4xx / 5xx errors • p95 latency

BUSINESS + DB Tickets sold • Seat holds • Inventory • DB health, query time + pool

MONITORED SERVICES

EXPORTED / PROBED TARGETS

01 **Grafana**

02 **Alertmanager**

03 **Loki**

04 **Grafana Alloy**

05 **Blackbox Exporter**

FAHAD ULLAH / OBSERVABILITY

Grafana Alloy → Loki

Collect, store and explore Docker container logs in Grafana

LOG COLLECTION PIPELINE

DOCKER COMPOSE / TICKETING

01 DOCKER
Container logs
stdout / stderr

→

02 ALLOY
Auto-discovery
Docker socket

→

03 LOKI
Store + search
TSDB /loki

→

04 GRAFANA
Logs dashboard
Recent logs

SCOPE	Only containers in Compose project ticketing
LABELS	container / compose_service / environment

COLLECTION & ACCESS

WHAT EACH LAYER DOES

ALLOY
Discovers ticketing containers and forwards their logs

LOKI
Receives logs on port 3100 and stores them persistently

GRAFANA
Queries Loki to inspect recent app and service logs

AWS CONFIGURATION Dedicated monitoring EC2 • private services • Grafana access through SSM port forwarding

FAHAD ULLAH / OBSERVABILITY

Blackbox Exporter + Alertmanager

Endpoint checks and application alerts delivered through one monitoring pipeline

AVAILABILITY CHECKS + ALERT DELIVERY

FLASK BACKEND / ENDPOINT PROBES

TARGETS **/healthz** **/readyz** HTTP 2xx • IPv4 • 5s

01 **BLACKBOX**
Probes endpoints

02 **PROMETHEUS**
Collects /probe

03 **ALERT RULES**
Checks conditions

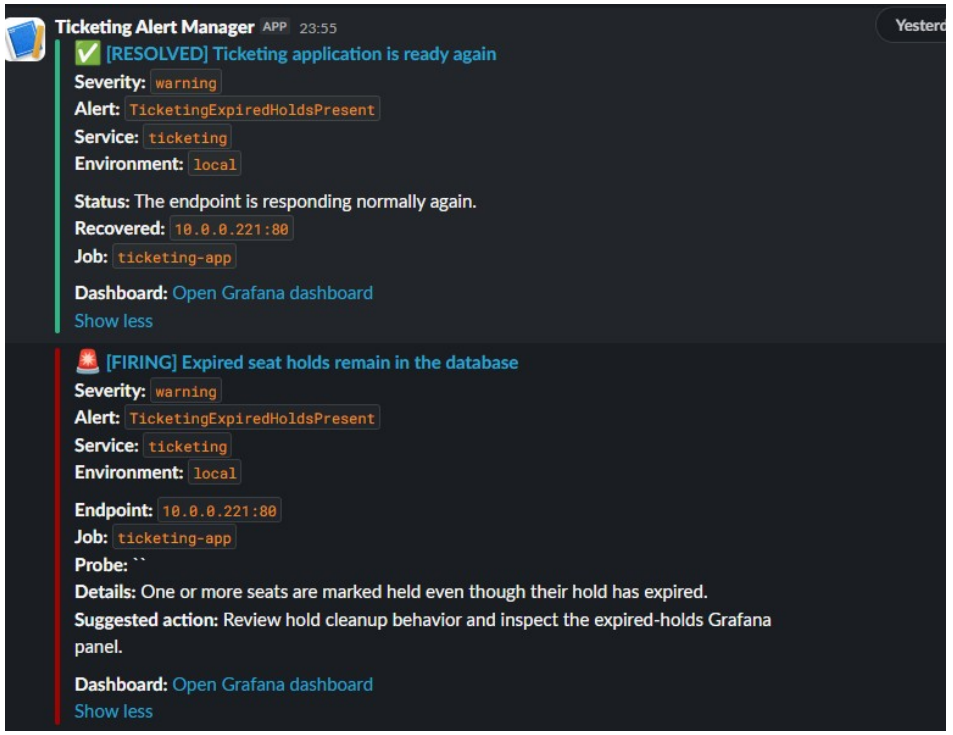
04 **ALERTMANAGER**
Routes to Slack

ALERT CONDITIONS

Health / readiness > 2 min

Response time > 1s for 5 min

REAL SLACK ALERTS / LOCAL ENVIRONMENT



FAHAD ULLAH / BACKEND + OBSERVABILITY

Checkout API + real ticket sales

Before, Grafana showed seat availability and holds, but checkout did not register a sale.

BEFORE / WHAT THE APP DID

Fake order reference

Frontend simulated a purchase
No real order or ticket was created
Grafana displayed holds, not sales

AFTER / WHAT I ADDED

POST /api/v1/events/<event_id>/checkout

01 VALIDATE
Customer + unexpired hold

02 COMMIT
Order, tickets + sold seats

03 COUNT
Sales metric after commit

CHECKOUT → MONITORING

FRONTEND → FLASK API → POSTGRESQL → PROMETHEUS → GRAFANA

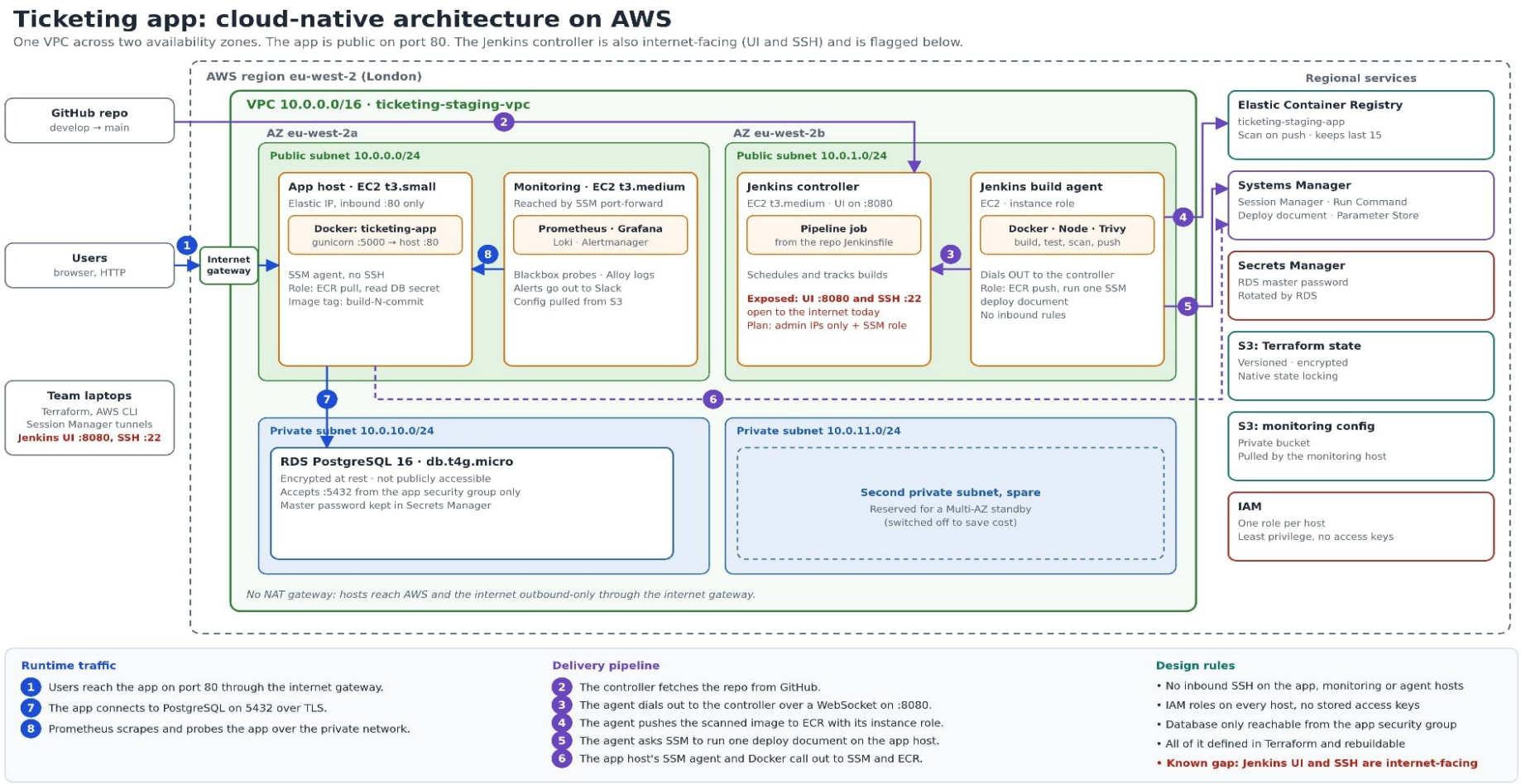


HOW CHECKOUT DATA FLOWS

01	FRONTEND	Sends seat IDs + customer
02	FLASK API	Validates hold and checkout
03	POSTGRESQL	Saves order + tickets; marks sold
04	PROMETHEUS	Scrapes Flask /metrics
05	GRAFANA	Charts seat status + sales

SAMUEL UDOM / AWS INFRASTRUCTURE

Ticketing app: cloud-native architecture on AWS



INFRASTRUCTURE AS CODE

SAMUEL UDOM / AWS INFRASTRUCTURE

If it is not in Terraform, it does not exist

A reviewable definition of the AWS platform, designed to be rebuilt rather than recreated by hand.

WHAT THE CODE DEFINES

NETWORK	VPC, public/private subnets and security groups
DATA	RDS PostgreSQL and its access boundaries
COMPUTE	App, Jenkins and monitoring EC2 hosts
IDENTITY & DELIVERY	IAM roles, ECR and the SSM deploy document

THE SAFE CHANGE CYCLE

1. **plan**
2. **review**
3. **apply**

STATE & DRIFT

- Versioned, encrypted S3 remote state
- Native state locking prevents concurrent applies
- plan -detailed-exitcode checks for drift
- -refresh-only shows out-of-band changes

BOOTSTRAP FIRST The state bucket is created separately; Terraform cannot store its own state before the bucket exists.

SECURITY & IDENTITY

Roles, not keys. Least privilege, not admin.

Four layers of control: identity, access, network and data.

01

IDENTITY

- One IAM role per host
- No long-lived access keys stored on servers
- IMDSv2 required for instance credentials

02

ACCESS

- SSM Session Manager for app, monitoring and agent access
- No inbound SSH on those hosts
- Agent connects outbound to controller over WebSocket

03

NETWORK

- Security groups reference other security groups
- RDS accepts :5432 from the app group only
- App monitoring traffic allowed from monitoring group

04

DATA

- RDS encrypted at rest
- Database password stored in Secrets Manager
- Trivy scans repository and container image

KNOWN GAPS Jenkins controller still uses SSH key access on port 22; its root volume is unencrypted. ECR tags are mutable, and remaining Trivy HIGH findings are logged but not yet fixed.

The agent role is deliberately narrow: push to one ECR repo, invoke one SSM document on one instance, and read only its required parameter.

SAMUEL UDOM / AWS INFRASTRUCTURE

Jenkins: the controller manages, the agent builds

01

CONTROLLER

The manager

- Runs the web UI on port 8080
- Holds jobs, plugins, users and build history
- Watches the repo and schedules builds
- Delegates execution to the agent

02

AGENT

The worker

- Checks out code; runs lint and tests
- Builds Docker images and runs Trivy
- Pushes to ECR using its instance role
- Starts SSM deployment and reports back

AGENT → CONTROLLER • OUTBOUND WEBSOCKET ON :8080 • NO INBOUND PORTS ON AGENT

RESILIENCE

Heavy builds do not bring down the Jenkins UI.

LEAST PRIVILEGE

Agent role pushes one repo and invokes one deploy document.

REPLACEABLE

Rebuild the agent without losing the pipeline setup.

SAMUEL UDOM / AWS INFRASTRUCTURE

Monitoring on its own instance, reached privately

WHY A SEPARATE INSTANCE

Monitoring survives the failure it reports.

- App outages are still visible and alertable
- Traffic spikes cannot starve Grafana or Prometheus
- Separate IAM role and security group limit exposure

HOW WE REACH IT

SSM port forwarding, no public ports.

- Start an SSM session from the laptop
- Open Grafana at localhost over the tunnel
- No public inbound rules or login page

MONITORING SERVICES

PROMETHEUS

App metrics

BLACKBOX

External probes

ALERTMANAGER

Slack alerts

GRAFANA + LOKI

Dashboards + logs

SAMUEL UDOM / NEXT IMPROVEMENTS

What we would improve with more time

01 SECURITY

- Restrict Jenkins UI and SSH to admin IPs
- Add controller SSM role, then close SSH
- Encrypt disk, immutable ECR tags, non-root container

02 PIPELINE

- Add Python unit and integration tests
- Production stage with manual approval
- Manage Jenkins configuration as code

03 RELIABILITY

- RDS Multi-AZ and longer backups
- Multiple app hosts behind a load balancer
- Private subnets with VPC endpoints

04 OBSERVABILITY

- Collect app container logs with Alloy
- Add node_exporter host metrics
- Alert on pipeline failures, not only app health